A3: Advanced Automated Analysis for Python

Barrier Certificates Meet Agentic LLM Triage

Halley Young

RiSE Group — Microsoft Research

2026

The Problem: Static Analysis Alert Fatigue

State of Python static analysis:

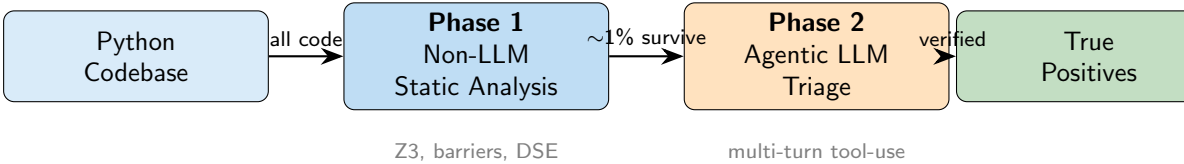
- Existing tools (Bandit, Semgrep, Pylint) produce **90–99% false positives**
- Developers **ignore alerts** $\Rightarrow$ real bugs ship
- Security teams spend days triaging noise

Root cause:

- Pattern-matching without semantic understanding
- No path feasibility reasoning
- No guard detection (e.g., `if x is not None`)

Typical tool output

A3's Approach: Two-Phase Architecture



- **Phase 1:** Formal methods filter the majority of false positives automatically
- **Phase 2:** An LLM agent *explores the codebase* to classify survivors
- **Result:** High precision — dramatically reduced noise

Codebase Architecture

```
a3_python/
  cli.py # CLI entry point
  analyzer.py # Core engine
  frontend/ # Bytecode loading
  cfg/ # CFG + call graph
  semantics/ # Crash summaries
  z3model/ # Z3 value/heap
  unsafe/ # Bug predicates
  contracts/ # Taint src/sink
  dse/ # Concolic exec
  barriers/ # Certificate synth
  ci/ # CI integration
    sarif.py # SARIF output
    baseline.py # Ratchet
    agentic_triage.py
    templates/ # GH Actions
```

Dependencies:

- Core: `z3-solver` only
- CI: + `anthropic, openai, pyyaml`
- Python $\geq$ 3.11 (bytecode format)

Design principles:

- Zero mandatory LLM dependency
- Formal methods first, LLM second
- SARIF as interchange format
- Modular: each `barriers/*.py` is an independent proof strategy

Phase 1: The 7-Step Symbolic Execution Pipeline

1. **Call Graph Construction** — whole-program CG from all `.py` files
2. **Crash Summary Computation** — bytecode disassembly $\rightarrow$ preconditions Π_f, effects E_f, exception sets X_f, nullability N_f
3. **Symbolic Model Construction** — Z3 representations: tagged values, heap model, taint lattice
4. **Guard Detection** — symbolic constraint extraction from `if`/`try`/`assert`/`isinstance`
5. **Z3 Symbolic Execution** — path feasibility via SMT satisfiability
6. **Staged Portfolio ("Kitchensink")** — barrier certificates, k-induction, interpolation, IC3/PDR, CEGIS *in parallel*
7. **Classification** — separate production code from test code

Key: Steps 4–6 are the formal-methods core.

Step 1–2: Bytecode Frontend & Call Graph

Frontend: Uses Python's built-in `compile()` → `types.CodeType`

- Disassembles bytecode instructions (CPython 3.11+)
- No AST heuristics — operates on the *actual* compiled representation

Call Graph: $G_{\text{call}} = (V, E)$

- V = all functions (from bytecode `co_consts`)
- E = call edges from `CALL` / `CALL_FUNCTION` instructions

CFG per function:

- `BasicBlock`: id, offsets, instructions, successors
- Edge types: `FALLTHROUGH`, `JUMP`, `COND_TRUE`, `COND_FALSE`, `EXCEPTION`
- Exception regions with handler locations and nesting depth

Barrier Certificates: Safety Without Exhaustive Search

Idea: Construct a function $B : S \to \mathbb{R}$ that separates safe from unsafe states.

Barrier Certificate Conditions

$$\begin{aligned} \textbf{Init:} \quad & \forall s_0 \in S_0.\ B(s_0) \geq \varepsilon \\ \textbf{Inductive:} \quad & \forall s.\ B(s) \geq 0 \implies B(s') \geq 0 \\ \textbf{Unsafe:} \quad & \forall s \in U.\ B(s) \leq -\varepsilon \end{aligned}$$

If all three hold, the unsafe region is provably unreachable.

- **Templates:** Linear, quadratic, polynomial, disjunctive regions
- **Synthesis:** Enumerate templates; validate with Z3; first inductive certificate wins
- **Specialized variants:** Hybrid (multi-mode), stochastic (supermartingale/Itô), SOS, ranking functions

Barrier Synthesis: Template Enumeration

11 template phases (tried in increasing complexity):

1. Constant barriers (trivial)
2. Single-variable linear: $c_0 + c_1 x$
3. Stack depth barriers
4. Loop range barriers
5. Collection size barriers
6. Progress measures (termination)
7. Two-variable linear: $c_0 + c_1 v_1 + c_2 v_2$
8. Quadratic: $ax^2 + bx + c$
9. Bivariate quadratic: $x^2 + y^2 \leq R^2$
10. Cubic polynomials
11. Piecewise / disjunctive regions

Algorithm:

1. For each template T_i in order of complexity:
2. Instantiate $B = T_i$ with symbolic coefficients $\vec{c}$
3. Encode Init $\wedge$ Inductive $\wedge$ Unsafe conditions as Z3 assertions
4. If SAT: extract $\vec{c} \Rightarrow$ **certificate found**, stop
5. If UNSAT or timeout: try next template

Guard-to-Barrier: From Code Patterns to Proofs

Key insight: User guards in code map directly to barrier certificates.

Guard Pattern	Barrier B	Bug Type
`if x is not None`	$B(x) = (x \neq \texttt{None})\ ?\ 1 : -1$	NULL_PTR
`if xs` (truthiness)	$B(\texttt{xs}) = \texttt{len(xs)} - 1$	BOUNDS
`if b != 0`	$B(b) = \lvert b \rvert - \varepsilon$	DIV_ZERO
`if k in d`	$B(k, d) = (k \in d)\ ?\ 1 : -1$	KEY_ERROR
`if i < len(xs)`	$B(i, \texttt{xs}) = (i \in [0, \texttt{len}))\ ?\ 1 : -1$	BOUNDS

Validation: Each translated barrier is checked for inductiveness by Z3.
Advantage: Replaces ad-hoc pattern matching with formal proofs.

Z3 Symbolic Model of Python Values

Tagged symbolic values: `SymbolicValue(tag, payload, taint)`

Value tags:

- `NONE, BOOL, INT, FLOAT`
- `STR, LIST, TUPLE, DICT`
- `OBJ, SLICE, ELLIPSIS`

Payload: Z3 `IntSort/RealSort/ObjId`

Heap model:

- `SeqLen(obj_id)` — Z3 function
- `HasKey(dict_id, key)` — Z3 function
- `DictSize(obj_id)` — Z3 function
- Sequence: length + element map
- Taint lattice per value

Example constraint:

$$\texttt{tag} = \texttt{LIST} \ \wedge \ \texttt{SeqLen(id)} \geq 0 \ \wedge \ \texttt{idx} < \texttt{SeqLen(id)} \implies \text{BOUNDS safe}$$

Guard Detection: Proving Bugs Unreachable

A3 recognizes when bugs are already guarded in user code:

```
def safe_divide(a, b):
    if b != 0: # guard: b != 0
        return a / b # DIV_ZERO unreachable!
    return 0.0
```

Symbolic reasoning:

1. Extract guard constraint: $b \neq 0$
2. Bug condition: $b = 0$
3. Check $\text{SAT}(b \neq 0 \ \wedge \ b = 0) = \text{UNSAT}$
4. $\Rightarrow$ Proven false positive

Detected guard patterns: `if x is not None` · `if xs` (truthiness) · `if b != 0` · `if k in d` · `if i < len(xs)`

Directed Symbolic Execution (DSE)

Goal: Confirm or refute bug reachability via constraint solving.

Algorithm:

1. Enumerate paths through the CFG to the bug location
2. For each path, accumulate a **path condition** $\mathrm{PC} = \bigwedge_i g_i$
3. Conjoin the **bug condition** ϕ_{bug}
4. Query Z3: $\mathrm{SAT}(\mathrm{PC} \wedge \phi_{\text{bug}})$?

If SAT:

- Extract concrete inputs from Z3 model
- **DSE-confirmed true positive**
- Provides a *witness* (counterexample)

If UNSAT:

- Bug is **provably unreachable**
- Classified as false positive
- No further investigation needed

Key data structure: `DSEResult` (status: realized|failed|error), with concrete inputs from Z3 model

Taint Analysis for Security Bugs

Taint lattice: Each `SymbolicValue` carries a set of taint labels.

Registered taint sources ($\tau = 1$):

- HTTP: `request.args`, `request.GET`
- User: `input()`, `sys.stdin`
- Env: `os.environ`, `os.getenv`
- Network: `urllib`, `requests`
- DB: `cursor.fetchone/all`

Registered taint sinks:

- `cursor.execute` → SQL_INJECTION
- `os.system/popen` → CMD_INJECTION
- `eval()/exec()` → CODE_INJECTION
- `pickle.loads` → UNSAFE_DESER
- HTML render → XSS

Status: Source/sink contracts and unsafe predicates are registered (48 security types). Taint propagation through the symbolic execution engine is a work in progress — currently, correctness bugs (NULL_PTR, DIV_ZERO, BOUNDS) are the primary detection targets.

Interprocedural Analysis via Crash Summaries

Each function gets a `CrashSummary`:

Π_f (`preconditions`)	Required parameter conditions (non-None, typed, bounded)
E_f (`may_trigger`)	Bug types function may trigger (+ `transitive_bugs`)
X_f (`may_raise`)	Exception types that may propagate
N_f (`param_nullability`)	Which params/returns may be `None`

Interprocedural propagation:

- Caller passes value violating callee's $\Pi_f \Rightarrow$ potential `NULL_PTR`
- Uncaught exception in $X_f \Rightarrow$ propagates to caller
- Guard in caller can satisfy callee's $\Pi_f \Rightarrow$ eliminates E_f

84 Bug Types Across Correctness & Security

Correctness (36 types):

- `DIV_ZERO`, `NULL_PTR`, `BOUNDS`
- `ASSERT_FAIL`, `UNBOUND_LOCAL`
- `INTEGER_OVERFLOW`, `FP_DOMAIN`
- `VALUE_ERROR`, `RUNTIME_ERROR`
- `NON_TERMINATION`, `STACK_OVERFLOW`
- `MEMORY_LEAK`, `USE_AFTER_FREE`
- `DATA_RACE`, `DEADLOCK`
- `TYPE_CONFUSION`, `NAME_ERROR`
- + 19 more (IO, file, module, etc.)

Security (48 types):

- **Injection:** SQL, Command, Code, Path, LDAP, XPath, NoSQL, Regex, Header, Cookie, Log
- **Web:** XSS, SSRF, CSRF, URL redirect
- **Crypto:** Weak crypto/key, broken algorithm, insecure protocol
- **Deser:** Pickle/YAML/XML unsafe loads, XXE
- **Secrets:** Cleartext logging/storage, hardcoded creds
- **ReDoS:** Polynomial/exponential regex

Each type has a registered **unsafe predicate** and **barrier template** for synthesis.

Unsafe Predicates: Formal Bug Definitions

Each bug type is defined by a **predicate** on the symbolic state:

Registry: `UNSAFE_PREDICATES: dict[bug_type → (predicate, extractor)]`

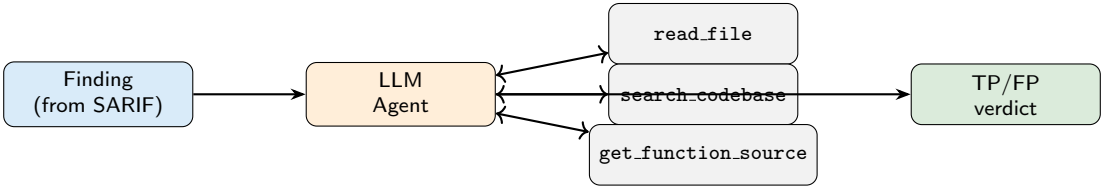
Bug Type	**Predicate** $P(\sigma)$	**Extractor**
DIV_ZERO	$\texttt{divisor}(\sigma) = 0$	divisor value
NULL_PTR	$\texttt{tag}(\sigma.v) = \texttt{NONE} \wedge \texttt{deref}(\sigma)$	variable name
BOUNDS	$\texttt{idx}(\sigma) \geq \texttt{SeqLen}(\sigma.s)$	index, length
SQL_INJ	$\tau(\sigma.q) = 1 \wedge \neg\texttt{Param}(\sigma)$	tainted query
CMD_INJ	$\tau(\sigma.\text{arg}) = 1 \wedge \texttt{at_shell_sink}$	command string

Four predicate categories:

1. **Correctness** (36): DIV_ZERO, NULL_PTR, BOUNDS, etc.
2. **Security** (48): injection, crypto, deser, secrets, ReDoS

Phase 2: Agentic LLM Triage

For findings that survive formal verification:



Tools available: `read_file`, `search_codebase`, `get_function_source`, `get_imports`, `list_directory`, `classify`

- **Not a one-shot prompt** — multi-turn, iterative investigation (2–6 tool calls)
- Agent reads callers, checks tests, follows imports, verifies guards
- Calls `classify(verdict, confidence, rationale)` when ready

Three Verdicts

Verdict	Meaning	Fraction
FP (proven)	Barrier certificate or DSE proves unreachability	majority
TP candidate	No proof found; sent to LLM triage	remainder
DSE-confirmed TP	Z3 found satisfying input triggering the crash	rare

Confidence scoring:

- Each finding gets a confidence score $c \in [0, 1]$
- Barrier proof $\Rightarrow$ c reduced toward 0 (FP)
- DSE witness $\Rightarrow$ c increased toward 1 (TP)
- LLM agent provides independent confidence + rationale

Agentic Investigation: A Concrete Trace

Finding: `NULL_PTR` in `get_user_email()` at line 42

Turn 1: Agent receives finding + function source

Turn 2: `search_codebase("get_user_email")` — finds 3 callers

Turn 3: `read_file("views.py", 100, 120)` — caller checks `if user:`

Turn 4: `get_function_source("validate_user")` — confirms None check

Turn 5: `classify(verdict="FP", confidence=0.9, rationale="All callers guard against None")`

Key design decisions:

- Max **10 turns** per finding (Turn 9: "call `classify` NOW")
- Tool output truncated to **6000 chars**
- Uses OpenAI function-calling format (converted for Anthropic)
- Providers: GitHub Models (free), OpenAI, Anthropic

Demo 1: NULL_PTR Detection & Guard Elimination

Unsafe version:

```python
def authenticate(username, db):
    record = db.get(username)
    # record could be None!
    return record['password_hash']
    # => NULL_PTR reported
```

```
$ a3 scan unsafe.py
NULL_PTR: authenticate
  unsafe.py:1 Confidence: 0.76
Total bugs found: 1
```

Safe version:

```python
def authenticate(username, db):
    record = db.get(username)
    if record is not None:
        return record['password_hash']
    return None
```

```
$ a3 scan safe.py
Total bugs found: 0
```

Z3:

$\text{SAT}(\texttt{record} \neq \texttt{None} \ \wedge \ \texttt{record} = \texttt{None}) = \text{UNSAT} \checkmark$

Demo 2: DIV_ZERO with Guard Detection

```python
def ratio(a, b):
    return a / b # DIV_ZERO: b could be 0
```

```
$ a3 scan ratio.py
DIV_ZERO: ratio
  ratio.py:1 Confidence: 0.84
Total bugs found: 1
```

Now add a guard:

```python
def ratio(a, b):
    if b != 0:
        return a / b # Now proven safe
    return 0.0
```

```
$ a3 scan ratio_safe.py
Total bugs found: 0
```

Z3 reasoning: Guard $b \neq 0$ makes bug condition $b = 0$ unsatisfiable at division site.

Demo 3: NULL_PTR — Chained Attribute Access

Unsafe:

```
def get_host(config):
    return config.database.host
    # NULL_PTR: config or
    # config.database could be None
```

```
$ a3 scan app.py
NULL_PTR: get_host
  app.py:1 Confidence: 0.76
Total bugs found: 1
```

Safe:

```
def get_host(config):
    if config is not None \
       and config.database is not None:
        return config.database.host
    return "localhost"
```

```
$ a3 scan app_safe.py
Total bugs found: 0
```

Guard detected: None-checks eliminate NULL_PTR at each dereference.

Demo 4: Multiple Bug Types in One Function

```python
def process_record(records, index, divisor):
    record = records[index] # BOUNDS
    value = record['amount'] / divisor # DIV_ZERO
    return value
```

```
$ a3 scan example.py
BOUNDS: process_record example.py:1 Confidence: 0.76
DIV_ZERO: process_record example.py:1 Confidence: 0.84
Total bugs found: 2
```

Add guards for both:

```python
def process_record(records, index, divisor):
    if records and index < len(records) and divisor != 0:
        record = records[index]
        value = record['amount'] / divisor
        return value
    return None
```

```
$ a3 scan example_safe.py
Total bugs found: 0
```

Demo 5: BOUNDS — Truthiness Guard

Unsafe:

```
def first_element(items):
    return items[0]
    # BOUNDS: items could be empty
```

```
$ a3 scan head.py --min-confidence 0
BOUNDS: first_element
  head.py:1 Confidence: 0.53
Total bugs found: 1
```

Safe:

```
def first_element(items):
    if items:
        return items[0]
    return None
```

```
$ a3 scan head_safe.py --min-confidence 0
Total bugs found: 0
```

Guard: Python truthiness check (`if items`) detected as non-empty guard for list access.

Demo 6: DIV_ZERO — Truthiness vs Explicit Guard

```python
def calculate_average(scores):
    total = sum(scores)
    return total / len(scores) # DIV_ZERO if scores is empty
```

```
$ a3 scan avg.py
DIV_ZERO: calculate_average avg.py:1 Confidence: 0.84
Total bugs found: 1
```

Guard with truthiness check (detected):

```python
def calculate_average(scores):
    if not scores:
        return 0.0
    return sum(scores) / len(scores)
```

```
$ a3 scan avg_safe.py
Total bugs found: 0
```

Note: `if not scores` is detected as a guard; `if len(scores) == 0` currently is not (direct `len()` comparison is a known gap).

Proof Strategy Portfolio ("Kitchensink")

Multiple verification techniques run **in parallel** — first proof wins:

Strategy	**Strength**	**Reference**
Barrier certificates	Continuous safety	Prajna & Jadbabaie, 2004
k-induction	Bounded loops	Sheeran et al., 2000
IC3/PDR	Property-directed	Bradley, 2011
Craig interpolation	Refinement	McMillan, 2003
CEGIS	Synthesis	Solar-Lezama, 2008
SOS (Positivstellensatz)	Polynomial	Parrilo, 2003
Ranking functions	Termination	Cook et al., 2006
Houdini	Annotation inference	Flanagan & Leino, 2001

Why portfolio? Different proof strategies excel on different code patterns. Running them in parallel gives best coverage without strategy selection.

Zero-Config CI Integration

```
$ a3 init . --copilot
$ git add .github/ .a3.yml .a3-baseline.json
$ git push
```

What happens on every push/PR:

1. Non-LLM scan → SARIF
2. Agentic triage (uses `GITHUB_TOKEN` — free, zero config)
3. Baseline ratchet check (new bugs ⇒ CI fails)
4. SARIF uploaded to GitHub Code Scanning dashboard

Baseline ratchet for incremental adoption:

- New findings not in baseline ⇒ CI fails
- Findings that disappear ⇒ auto-pruned
- Pre-existing issues ⇒ ignored (never blocked on legacy debt)

Formal Semantics: The Python Subset

What A3 models:

- **Values:** `None`, booleans, integers (unbounded), floats, strings (length-abstracted)
- **Collections:** Lists, tuples, dicts — modeled as (length, element_map)
- **Control flow:** `if/else`, `for/while`, `try/except/finally`, `with`
- **Functions:** First-class, closures, `*args`/`**kwargs`
- **Objects:** Attribute access, `isinstance`, method resolution

What A3 approximates:

- **Dynamic dispatch:** Over-approximated in call graph (all possible targets)
- **Heap aliasing:** ObjId-based, not full shape analysis
- **Strings:** Length-tracked; content analysis only for taint
- **Generators/async:** Treated as ordinary functions

Soundness guarantee: Barrier proofs are sound *for the modeled semantics*. The gap between model and CPython is the source of unsoundness.

Evaluation on Real-World Python Projects

BugsInPy benchmark: 493 real bugs from 17 large Python projects (pandas, scikit-learn, flask, tornado, youtube-dl, black, . . .)

Phase 1 (static analysis):

- Filters guarded bugs as FP
- Barrier certificates prove guards
- DSE confirms subset as TP
- Runs in seconds per file

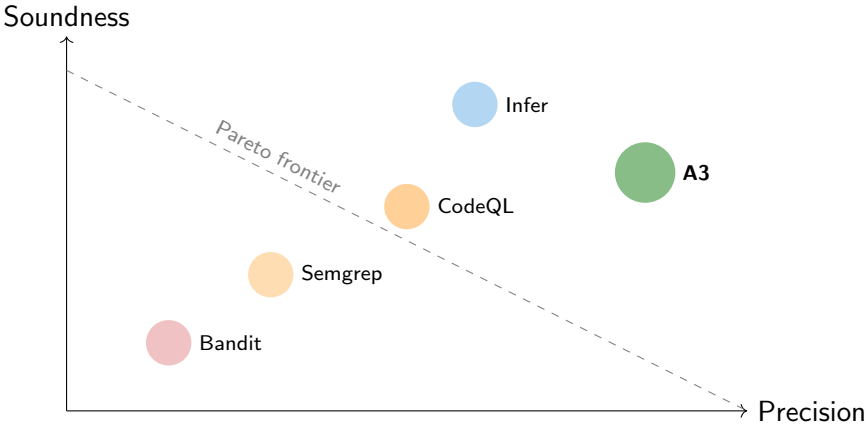
Phase 2 (agentic triage):

- Classifies unproven findings
- 2–6 tool calls per finding
- Agent reads callers, tests, imports
- High precision on survivor set

Key insight: The two phases are *complementary*:

- Phase 1 eliminates what *can* be proven formally
- Phase 2 handles the *semantic ambiguity* that formal methods cannot resolve

The Soundness–Precision Tradeoff



A3's position: Sacrifices full soundness for **very high precision**.
Barrier proofs are locally sound; the LLM fallback handles the rest.
Design principle: A tool developers *actually use* beats a sound tool they ignore.

Formal Methods FAQ (1/2)

Q: Is the analysis sound?
A: **No.** Soundness is explicitly traded for precision. The barrier proofs are sound *for the modeled semantics*, but the Python model is an approximation (no full heap semantics, dynamic dispatch is approximate).

Q: Is it complete?
A: No. Template enumeration is incomplete; some safe programs lack polynomial barriers. The portfolio mitigates this but cannot guarantee completeness.

Q: What about dynamic features (eval, monkey-patching)?
A: Conservatively flagged. Dynamic dispatch is over-approximated in the call graph. `eval`/`exec` are taint sinks (CODE_INJECTION).

Q: How does it scale?
A: Per-function analysis is linear in LOC. Interprocedural is bounded by call graph size. Z3 queries are typically small (10–100 constraints). Portfolio strategies have individual timeouts.

Formal Methods FAQ (2/2)

Q: Why barriers instead of abstract interpretation?
A: Barriers provide *per-bug* proofs (targeted) vs. whole-program invariants. Much cheaper for large codebases — we only need to prove specific properties unreachable.

Q: How do you handle loops?
A: Multiple strategies: k-induction for bounded loops, ranking functions for termination, barrier certificates for safety within loops. The portfolio tries all in parallel.

Q: What about the LLM — isn't that unsound by definition?
A: Yes. The LLM is *not* part of the formal reasoning. It only classifies findings that *already escaped* formal verification. Think of it as a high-quality human reviewer, not a proof engine.

Q: Can barrier proofs be checked independently?
A: Yes. A barrier certificate is a *witness*: given B, checking Init $\wedge$ Inductive $\wedge$ Unsafe is a Z3 query. Proof auditing is trivial compared to proof search.

Open Problems for the FM Community

1. **Heap-sensitive barriers:** Can barrier certificates reason about linked structures, aliasing, and shape properties in Python?
2. **Completeness of template enumeration:** What is the minimal template basis that covers "common" Python patterns? Is there a PAC-learning formulation?
3. **Compositional taint proofs:** Can we compose per-function barrier certificates to get whole-program taint-freedom guarantees?
4. **Certified Python semantics:** A mechanized (Lean/Coq) semantics of CPython 3.11+ bytecode would let us prove soundness of the Z3 model.
5. **LLM-guided proof search:** Can an LLM suggest barrier templates or invariants that accelerate formal verification? (Neural certificate synthesis)
6. **Incremental re-verification:** Given a diff, which barrier proofs are invalidated? Can we re-verify only affected functions?

Related Work

Tool	Approach	Precision	Formal?
Bandit	Pattern matching	Low	No
Semgrep	Structural patterns	Medium	No
Pyright/mypy	Type checking	High (types only)	Partial
CodeQL	Dataflow queries	Medium-High	Partial
Infer	Separation logic	High	Yes
A3	**Barriers + DSE + LLM**	**Very High**	**Yes (Phase 1)**

Key differentiators:

- **Only Python-specific** barrier certificate tool
- **Portfolio of 8+ proof strategies** (not just one technique)
- **Agentic LLM fallback** for what formal methods can't prove
- **84 bug types** spanning correctness + security

Future Directions

- **Richer heap models:** Full shape analysis for Python containers
- **Neural barrier synthesis:** Learn barrier templates from code patterns
- **Incremental analysis:** Re-verify only changed functions on each commit
- **Cross-language:** Extend Z3 model to TypeScript/JavaScript
- **Formal soundness proofs:** Mechanize the Python subset semantics in Lean/Coq

Questions?

`pip install a3-python` — `a3 scan . --triage`

https://github.com/thehalleyyoung/a3-python